Bicol University - College of Science
Department of Information Technology

IT 106 – Application Development and Emerging Technologies
2nd Semester 2025-2026

Personal Activity Journal

Activity: **Group REST API Enhancement and Multi-Client Implementation**

Submitted by:

Cayla C. Villanueva
BSIT - 3B

April 17, 2026

Tasks Performed

I. User Flow Conceptualization

Considering the use cases of the database tables within the interface, I came up with the original user role flow. Users are designated as either teachers or students at the time of registration. The system offers features that are compatible with the database structure and API logic based on this role.

The interface enables teachers to manage courses and see which students are in each course. It also allows students to enroll in courses, view their enrolled courses, and access course details. This role-based flow keeps the system organized and limits each user to features that are relevant to their role, such as teachers being able to manage courses and students being able to enroll in them.

To ensure this process works with the system, I worked with Vince (UI/UX designer) and Justine (backend developer). We made sure the front end works as expected and supports the API endpoints and database relationships.

II. Frontend Development and API Integration

My primary task was to create the system's client-side interface. I used HTML, CSS, and JavaScript to create the student administration dashboard, login page, and registration page. These user interfaces were created to make data entry simple and system navigation effective.

I made forms for the registration and login pages that ask for information like account ID, name, and password. After that, the inputs were sent to the REST API using JavaScript functions. The API sent back JSON replies after processing the request, which I managed and displayed correctly on the interface.

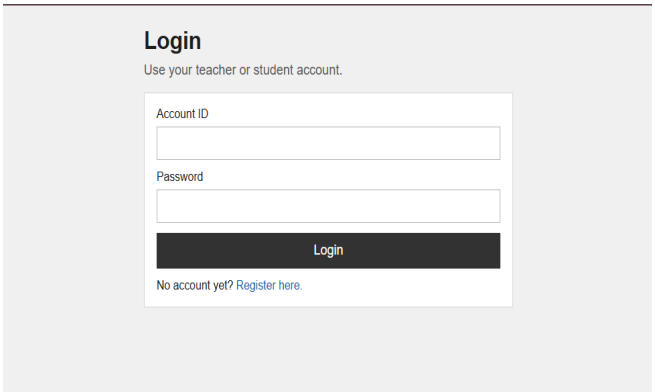

Figure 1. *Login Page Interface*

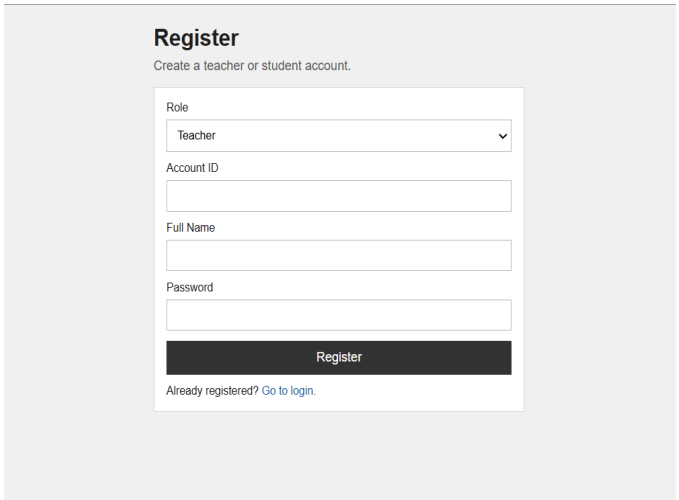

Figure 2. *Registration Page Interface*

In the student management dashboard, I added some of the functionalities, including being able to see the courses in which the student is enrolled and being able to access information relating to those courses. To ensure that the interface is responsive to the actions of the user, I made sure that the data on the frontend was always current in real-time according to the responses from the API.

I also made sure that the frontend supports the role-based behavior of the system by only showing the right features after login. This meant working with the backend to make sure that the API returns the right user roles and data.

For example, the frontend sends a request to the API using JavaScript to validate login credentials:

```
function login() {
    let account_id = document.getElementById("account_id").value;
    let password = document.getElementById("password").value;

    fetch(`http://localhost/service.php?action=login&account_id=${account_id}&password=${password}`)
        .then(response => response.json())
        .then(data => {
            document.getElementById("result").innerText = data.message;
        })
        .catch(error => {
            console.log(error);
        });
}
```

Figure 3. *Frontend API Request Implementation (Login Function)*

III. Integration Challenges and Refinement

During the development, I struggled quite a bit connecting to the API on the frontend. This is because the endpoints were wrong as well as the parameters that needed to be included. Once these issues were fixed, it resulted in many errors when requesting the data to present back to the user.

Once I was able to retrieve the data from the API, another obstacle I faced was formatting the JSON responses. These responses would return data that I needed to figure out how to display on the dashboard that would make sense to the user. Making the data presentable to the user required me to introduce some changes to the layout and format of the dashboard.

To overcome these issues, I had to work with Vince (UI/UX Designer) to figure out what UI elements to use and how to format the dashboard so that it matches the design. I had to work with our backend developer (Justine) to ensure we had the correct endpoints and parameters for the API. Both the frontend and API had to work together, so I ensured the frontend was able to connect to the API correctly.

```html
<section id="dashboardSection">
    <!-- Session summary and quick actions for the active user. -->
    <section class="panel toolbar">
        <div>
            <strong id="sessionName">Not logged in</strong>
            <div id="sessionMeta" class="muted"></div>
        </div>
        <button id="logoutButton" type="button">Logout</button>
    </section>

    <section class="panel">
        <div class="section-header">
            <h2>Courses</h2>
            <button id="refreshCoursesButton" type="button">Refresh</button>
        </div>
        <div id="courseList" class="output empty">No courses to display.</div>
    </section>
```

Figure 3. *Shared Dashboard*

This represents the main dashboard container of the browser-based client. It serves as the shared structure for the logged-in user interface, holding the session summary, course list, role-based action panels, and response display area.

```html
<section id="teacherSection" class="hidden">
    <section class="panel">
        <h2>Teacher Actions</h2>

        <div class="button-row action-switcher">
            <button id="showCreateCourseButton" type="button">Create Course</button>
            <button id="showViewCourseButton" type="button">View Course</button>
        </div>

        <form id="createCourseForm" class="form-card hidden top-spaced">
            <label>
                Course Code
                <input type="text" name="course_code">
            </label>

            <label>
                Course Name
                <input type="text" name="course_name">
            </label>

            <button type="submit">Save Course</button>
```

Figure 3. *Role-specific (teacher) dashboard section*

This figure shows the teacher-specific section of the dashboard. It includes controls to add and view courses and manage students within a course. It consists of input fields and action buttons that allow the user to perform operations (view all students, search for a student, remove a student from a course etc.)